MUHAMMAD NAWAZ SHAREEF UNIVERSITY OF ENGEENIRING AND TECHNOLOGY, MULTAN

LAB MANUAL

Web Technologies

Name

M. JUNAID

Reg. No

2023-IT-39

Degree

BS-IT

Section

B

Submitted to

MAM SEHRISH SALEEM

LAB-NO#01

❖ Objectives:

This lab introduces the brief difference between internet and world-wide-web (WWW), web browser and web server and also the fundamental structure of an HTML document and key elements for formatting and organizing content:

- Introduction to internet and world wide web(WWW).
- Introduction to web server and web browser.

1: Introduction to the Internet and World Wide Web (WWW)

Internet:

The Internet is a global network that connects billions of devices, enabling communication, data sharing, and access to online services. It operates using the TCP/IP protocol and supports various applications like email, cloud computing, and social media. Developed in the 1960s, the Internet has revolutionized industries worldwide.

World Wide Web (WWW):

The World Wide Web (WWW), invented by Tim Berners-Lee in 1989, is a system of interlinked web pages accessed via the Internet using HTTP/HTTPS protocols. It allows users to browse websites, retrieve information, and interact with digital content through web browsers and hyperlinks.

Key Difference:

- **Internet:** The infrastructure that connects networks globally.
- **WWW:** A service on the Internet that provides access to websites and digital content.

2: Introduction to Web Browser and Web Server

Web Browser:

A web browser is a software application used to access and display websites on the World Wide Web (WWW). It retrieves web pages from a web server using HTTP/HTTPS protocols and renders them for users.

Examples: Google Chrome, Mozilla Firefox, Microsoft Edge, Safari.

Functions of a Web Browser:

- Fetches web pages from servers via URLs.
- Interprets and displays HTML, CSS, and JavaScript.
- Supports interactive elements like forms and multimedia.

Web Server:

A web server is a computer system or software that stores, processes, and delivers web pages to users upon request. It responds to browser requests and sends back the requested content.

Examples: Apache, Nginx, Microsoft IIS, Lite-Speed.

Functions of a Web Server:

- Hosts websites and manages HTTP requests.
- Processes dynamic and static content.
- Ensures security and data handling.

Key Difference:

- **Web Browser:** A client-side application used to view web content.
- **Web Server:** A server-side system that stores and serves web pages to browsers.

LAB-NO#02

❖ Objective:

- Introduction to HTML.
- Using heading tags (<h1> to <h6>) for structured text.

Introduction to HTML (Hyper-Text Markup Language)

What is HTML?

HTML (Hyper-Text Markup Language) is the standard language used to create and structure web pages. It defines the content and layout of a webpage using a system of tags and elements.

Key Features of HTML

Structure-Based: Defines headings, paragraphs, tables, lists, and more.

Uses Tags: Elements are enclosed in angle brackets (<>).

Supports Multimedia: Can embed images, videos, and audio.

Hyperlinking: Connects different web pages using <a> tags.

The basic structure of an HTML document and heading tags.

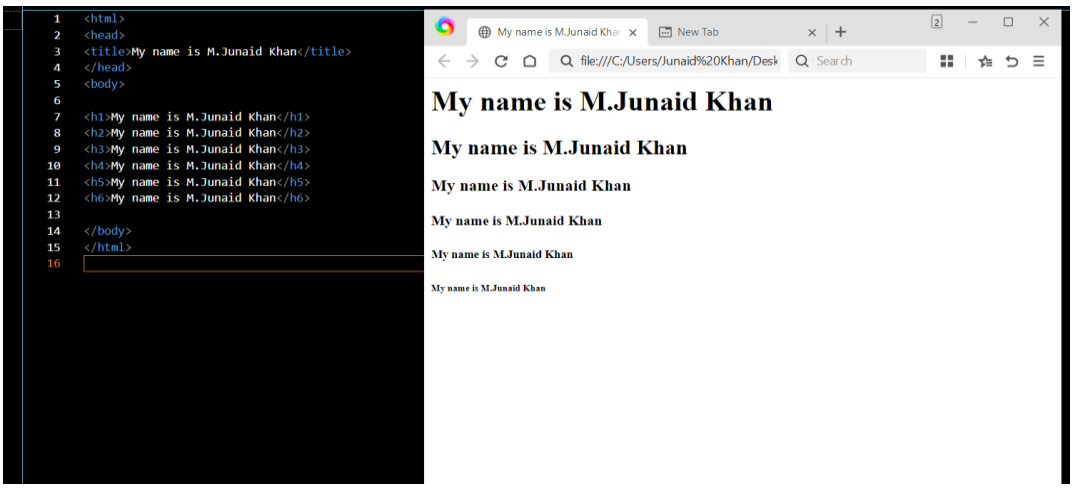
Theory:

HTML (Hyper-Text Markup Language) is the foundation of web development, used to create web pages. Every HTML document consists of essential tags that define its structure.

The basic structure of an HTML document includes:

- **<!DOCTYPE html> :** Defines the document type
- **<html> :** The root element
- **<head> :**Contains meta-information about the document
- **<title> :**Sets the page title (appears on the browser tab)
- **<body> :** Contains the visible content of the webpage

Code Example:



Explanation:

- **<h1> to <h6>:** These are heading tags used to define headings in an HTML document.

- <h1> is the largest heading, while <h6> is the smallest.
- Headings are important for structuring content and improving SEO (Search Engine Optimization).

LAB-NO#03

❖ Objective:

- ➢ Creating tables to display data in a grid format.
- ➢ Modifying background and text colors with attributes and inline styling.

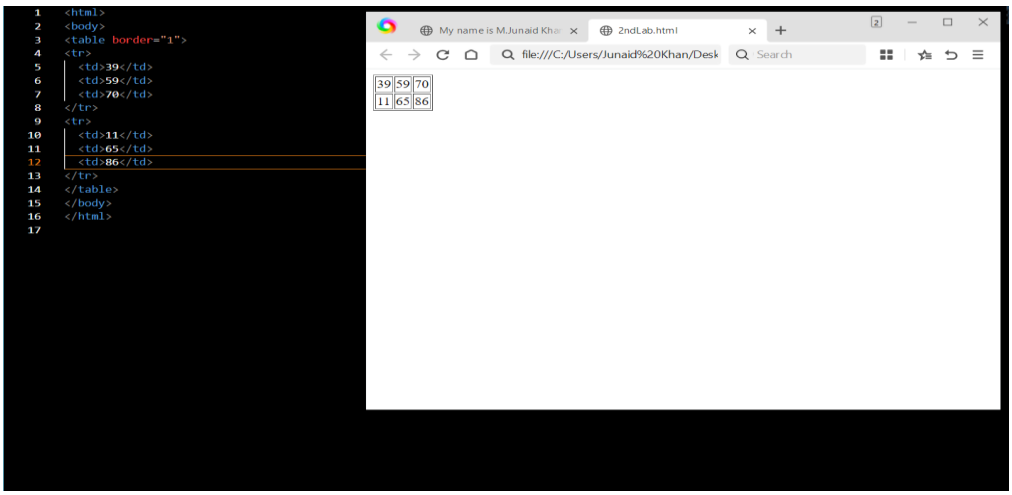
Tables in HTML to display data in a structured grid format.

Theory:

HTML tables allow you to present information in rows and columns. Here are the key elements used to build a table:

- **<table>:** Defines the table.
- **Border attribute:** Specifies the border width of the table. (Note: Using CSS is generally preferred for styling, but the border attribute is useful for quick demonstrations.)
- **<tr>:** Defines a table row.
- **<td>:** Defines a table cell (data).

Code Example:



Explanation:

- **<!DOCTYPE html>:** Declares the document type.
- **<html>:** The root element of the HTML page.
- **<body>:** Contains the content that is displayed on the page.
- **<table border="1">:** Creates a table with a border width of 1 pixel.
- **<tr>:** Each <tr> element defines a new row in the table.
- **<td>:** Each <td> element defines a cell within a row. The content inside these tags is the data that appears in the table cell.

Background and text colors using HTML attributes and inline styling.

Theory:

Background Color:

The <body> tag can have a bgcolor attribute that sets the background color of the entire webpage.

In this example, bgcolor="Dark Choclate" sets the background color. Note that using standard CSS is a more modern approach, but HTML attributes are useful for quick demonstrations.

Text Color and Font Styling:

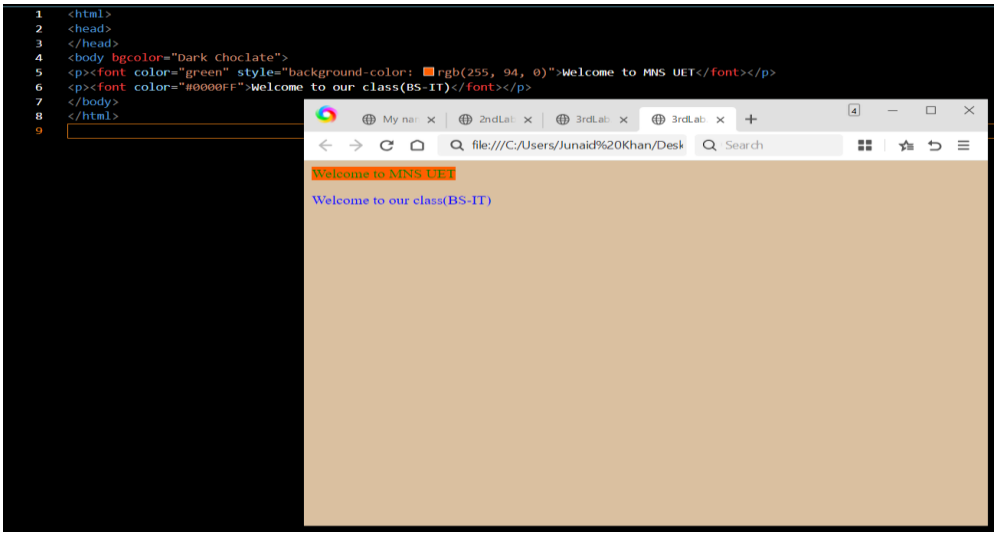
The <font> tag is used to change the text color and font properties. Although it's considered deprecated in favor of CSS, it can still be used for basic styling.

The color attribute sets the color of the text. For example, color="green" will render the text in green.

Inline styling using the style attribute allows for additional CSS properties to be applied. In this case, style="background-color: rgb(255, 94, 0)" sets the background color for that specific text.

The color value can also be set using hexadecimal values, such as color="#0000FF", which represents blue.

Code Example:



Explanation:

- **<!DOCTYPE html>:** Declares the document type and version of HTML.
- **<html>:** Root element of the HTML document.
- **<head>:** Contains metadata about the document, such as the <title>.
- **<title>:** Sets the title of the webpage (appears in the browser tab).
- **<body bgcolor="Dark Choclate">:** Applies a dark chocolate color as the background for the entire page.
- **<p>:** Defines paragraphs to separate content.
- **<font>:**The first <font> tag sets the text "Welcome to MNS UET" in green with an inline background color (rgb(255, 94, 0)).
- The second <font> tag sets the text "Welcome to our class(BS-IT)" to blue using a hexadecimal color code.

LAB-NO#04

❖ Objectives:

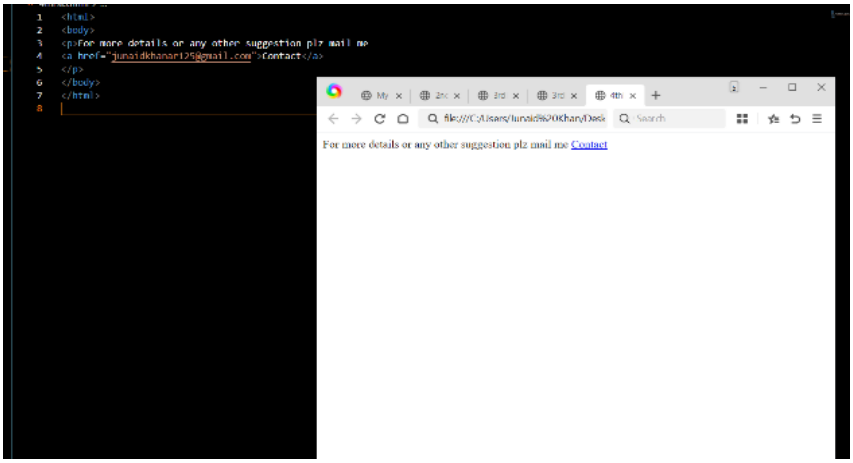
- ➢ Creating email hyperlinks for easy contact.
- ➢ Using the <hr> tag to separate content sections.

Email hyperlink in HTML to allow users to contact you with a single click.

Theory:

- The <a> (anchor tag) is used to create hyperlinks in HTML.
- The href attribute defines the destination of the link.
- To create an email link, use mailto: followed by the email address inside the href attribute.
- When a user clicks on the email link, their default email client (e.g., Gmail, Outlook) will open with a pre-filled recipient.

Code Example:



Explanation:

- **<p>:** Defines a paragraph containing the email link.
- **Contact:
- **The mailto:** protocol makes the link open an email client.
- Clicking the "Contact" link will open the user's default email application with junaidkhanar125@gmail.com already filled in the recipient field.

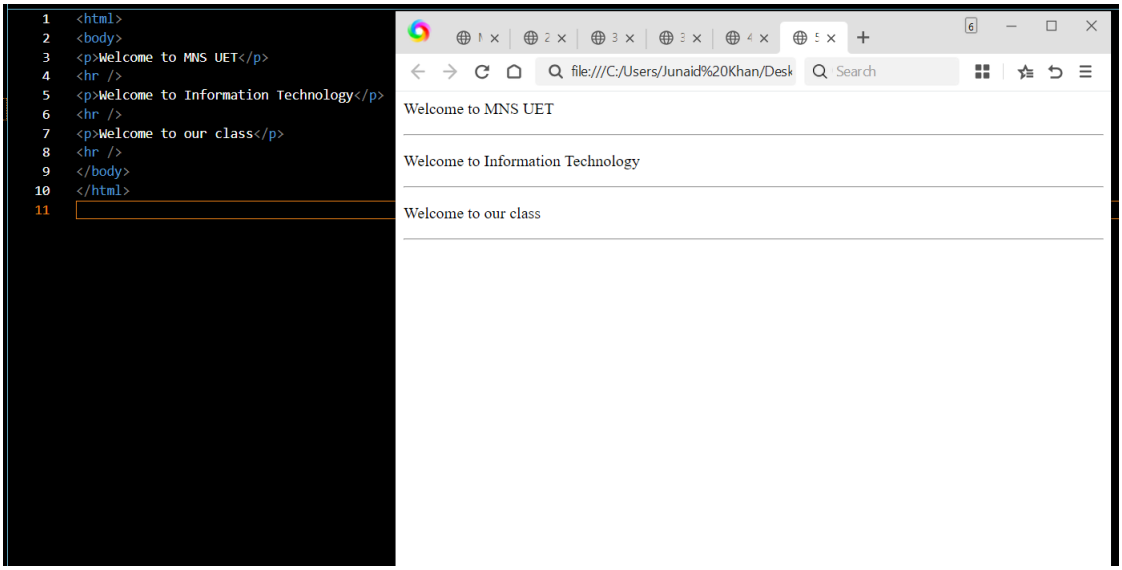
Use the <hr> (horizontal rule) tag to separate content sections in an HTML document.

Theory:

- The <hr> tag is an empty element (self-closing), meaning it does not require a closing tag.

- It is used to create a horizontal line, acting as a divider to separate different sections of a webpage.
- The <hr> tag is commonly used for readability and to visually distinguish sections.

Code Example:



Explanation:

- **<p>:** Defines a paragraph of text.
- **<hr />:** Creates a horizontal line that visually separates sections.
- The first <hr> separates "Welcome to MNS UET" from "Welcome to Information Technology."
- The second <hr> separates "Welcome to Information Technology" from "Welcome to our class."
- The third <hr> is used after the last paragraph to maintain a consistent design.

Lab-NO#05

❖ Objectives:

- To understand how to create and use different types of lists in HTML.

- To differentiate between **ordered**, **unordered**, and **description** lists.
- To use lists for structured and organized content display on web pages.

1. Unordered List (<ul>)

An unordered list in HTML is a list of items where the order does not matter. It is displayed with bullet points by default and is created using the <ul> tag with list items defined inside <li> tags.

Use Case: To show items like features, options, or collections where sequence isn't important.

Code:

unorderedlist.html > html > body > ul > li

```
<!DOCTYPE html>
<html>
<head>
    <title>Unordered List</title>
</head>
<body>

    <h2>Unordered List Example</h2>
    <ul>
        <li>Junaid</li>
        <li>Khan</li>
        <li>Bloch</li>
    </ul>

</body>
</html>
```

Output:

2. Ordered List (<ol>)

An ordered list in HTML displays a list of items in a specific sequence, using numbers (1, 2, 3) or letters (a, b, c). It is created using the <ol> tag and each list item is added with the <li> tag.

Use Case: Perfect for step-by-step instructions, ranking, or processes that follow an order.

Code:

```html
<!DOCTYPE html>
<html>
<head>
    <title>Ordered List</title>
</head>
<body>

    <h2>Ordered List Example</h2>
    <ol>
        <li>Register for the course</li>
        <li>Attend lectures</li>
        <li>Submit assignments</li>
    </ol>

</body>
</html>
```

Output:

3. Description List (<dl>)

A description list in HTML is used to define terms and their descriptions. It uses the <dl> tag as a container, where <dt> is used for the term and <dd> for the description or definition of the term.

Use Case: Ideal for glossaries, dictionaries, FAQs, or specifications.

Code:

```html
<!DOCTYPE html>
<html>
<head>
    <title>Description List</title>
</head>
<body>

    <h2>Description List Example</h2>
    <dl>
        <dt>JK</dt>
        <dd>Junaid Khan</dd>

        <dt>GB</dt>
        <dd>Gopang Bloch</dd>

        <dt>k</dt>
        <dd>Khan</dd>
    </dl>

</body>
</html>
```

Output:

LAB-NO#06

Working with Forms in HTML

❖ Objective:

- To learn how to create interactive web forms using HTML and understand various input types.

Explanation:

HTML Forms are used to collect user input. The data entered by users can be sent to a server for processing.

Key Tags in Forms:

<form>: Main container for form elements.

<input>: Used to take user input (text, password, radio, checkbox, etc.).

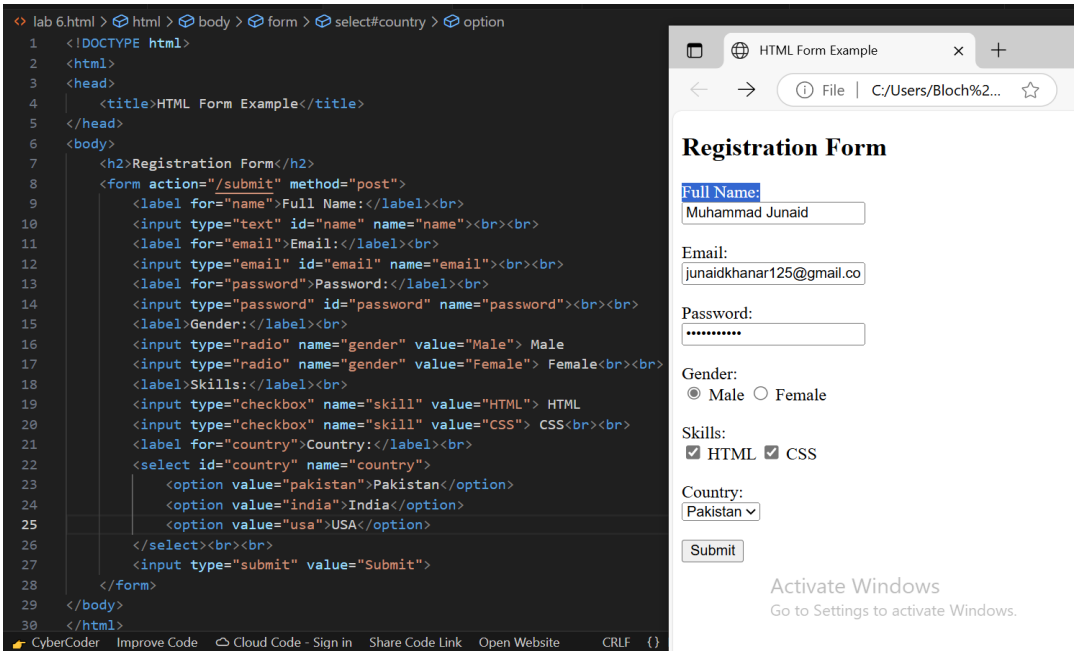
<label>: Describes the input field.

<textarea>: Multi-line text input.

<select> and <option>: Drop-down lists.

type attribute: Specifies the type of input (e.g., text, email, submit, etc.).

Code /Output Screen Shot:



LAB- NO#07

Adding Multimedia in HTML (Images, Audio, Video)

❖ Objective:

- To learn how to embed multimedia content such as images, audio, and video in an HTML webpage.

Explanation:

Multimedia elements make web pages more engaging. HTML provides specific tags to include different types of media:

Images: Displayed using the <img> tag.

src: Source of the image.

alt: Alternative text shown if the image doesn't load.

Audio: Added using the <audio> tag with the controls attribute to display play/pause buttons.

src: Path to audio file.

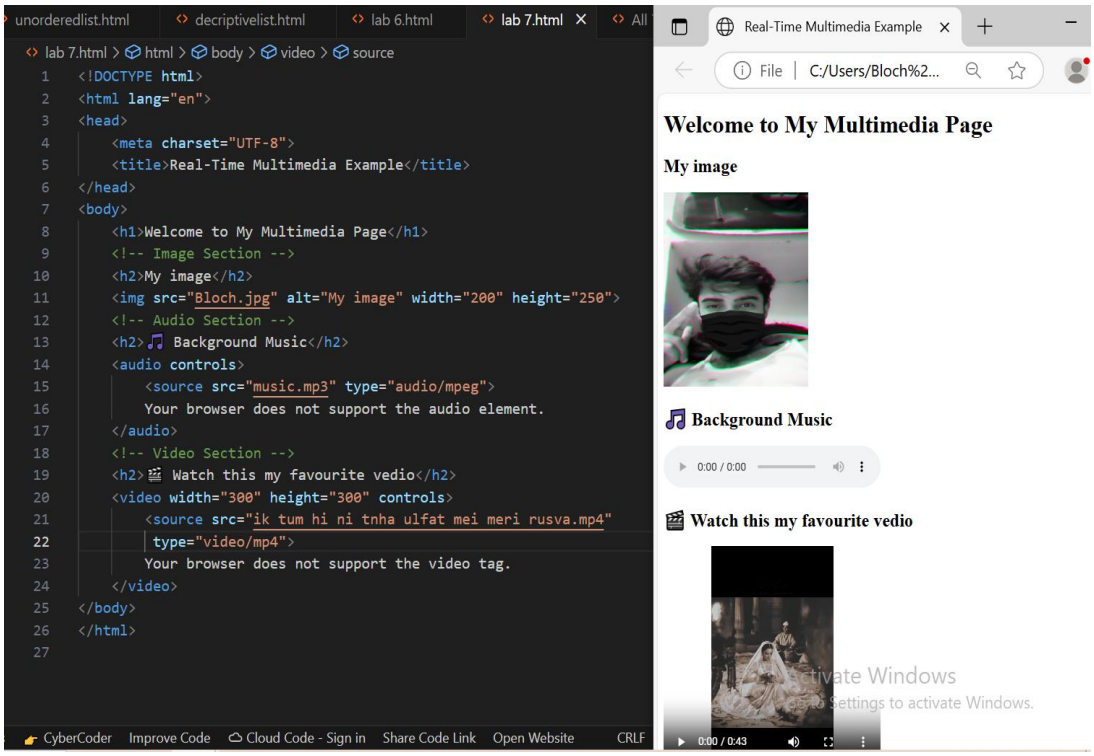
Supported formats: .mp3, .ogg, .wav.

Video: Embedded using the <video> tag with controls.

src: Path to video file.

Supported formats: .mp4, .webm, .ogg.

Code/Output Screen Shot:



LAB-NO#08

Basics of XML

❖ Objective:

- To learn the fundamentals of XML --- what it is, why it is used, and how to write a well-formed XML document with correct syntax and structure.

What is XML?

- XML stands for Extensible Markup Language. It is a markup language like HTML, but the purpose of XML is not to display data, it is to store and transport data in a structured and readable format.
- Unlike HTML, XML does not have predefined tags. You create your own tags based on the data you want to represent. This makes XML highly flexible and useful in many fields like data storage, configuration files, and web APIs.

Key Features of XML:

- XML is self-descriptive — it tells you what the data is.
- You can define your own tags to suit your data.
- It is both human-readable and machine-readable.
- XML is platform-independent and used to exchange data between different systems or software.
- XML must be well-formed, which means it must follow specific syntax rules.

Basic Syntax Rules in XML:

1. **Every opening tag must have a closing tag.**

Correct: <name>Junaid</name>

Incorrect: <name>Junaid

2. **XML is case-sensitive.**

<Name> and <name> are treated as different tags.

3. **There must be one root element that wraps all other elements.**

Example: <students> ... </students>

4. **Attributes must be enclosed in quotes.**

Example: <student id="101">

5. **Whitespace is preserved.**

If you write extra spaces or new lines, XML will consider them part of the data.

Important Terms Explained:

- **Element/Tag:** A pair like <name>Junaid</name> that holds your data.

- **Attribute:** Extra information inside a tag, such as <student id="S001">.
- **Root Element:** The first (and only) top-level tag that contains all others, e.g., <students>.
- **Nested Elements:** Elements inside other elements — like putting <name> inside <student>.
- **Well-Formed XML:** Properly structured XML that follows all syntax rules.
- **XML Declaration:** At the top of every XML file, we can declare the XML version and encoding like this:

<?xml version="1.0" encoding="UTF-8"?>

Example:

XMLEXMPLE.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>

<students>
    <student id="2023-IT-39">
        <name>Muhammad Junaid</name>
        <department>BS Information Technology</department>
        <university>MNS UET Multan</university>
    </student>

    <student id="2023-IT-65">
        <name>Ali Raza</name>
        <department>BS Computer Science</department>
        <university>MNS UET MULTAN</university>
    </student>
</students>
```

Explanation of the Code:

- **<?xml version="1.0"?>:** Declares the XML version.
- **<students>:** Root element that holds all student records.
- **<student id="2023-IT-39">:** A student element with an attribute id.

Inside each <student>, we have child elements:

- **<name>:** Student's name
- **<department>:** Their department
- **<university>:** The name of the university
- </students> closes the root element.

Where is XML Used?

- In web APIs (to send/receive data).
- In Android apps (layout files).
- In RSS feeds.
- For configuration files in software.
- In databases for structured data transfer.

LAB-NO#08

Introduction to CSS (Inline, Internal, External)

❖ Objective:

- To understand the basic concept of CSS and how to apply styles to HTML using three different methods:

Inline CSS, Internal CSS, and External CSS.

Explanation:

What is CSS?

- CSS (Cascading Style Sheets) is a language used to describe the style and layout of a webpage.
- HTML gives the structure, and CSS gives the look/design.

For example, if HTML is the body, CSS is the clothing and makeup.

Why Use CSS?

- To change colors, fonts, sizes, and layout.
- Makes your website attractive and user-friendly.
- Allows separation of content and design (you don't have to write style repeatedly).

Three Ways to Use CSS:

1. Inline CSS

- CSS is written inside the HTML tag using the style attribute.
- Best for applying style to one specific element only.

Syntax:

<tagname style="property: value;">Content</tagname>

Example:

This is red and large text.

- color: red; sets the text color.
- font-size: 20px; makes the text bigger.

2. Internal CSS

- CSS is written inside the <style> tag in the <head> section of the HTML file.
- Best when you're working on a single HTML file.

Syntax:

```
<!DOCTYPE html>
<html>
<head>
   <style>
      selector {
         property: value;
         property: value;
      }
   </style>
</head>
```

<body>

<!-- HTML content -->

</body>

</html>

Example:

- Inside <style>, we write CSS rules for elements.
- h1 is a selector, { color: green; } is the rule.
- text-align: center; centers the text.

3. External CSS

- CSS is written in a separate file with a .css extension (e.g., style.css).
- It is then linked to the HTML file using the <link> tag.
- Best for large projects with many HTML files.

Syntax:

HTML file:

<head>

<link rel="stylesheet" href="filename.css">

</head>

CSS file (filename.css):

```
selector {
    property: value;
}
```

Example:

Index html:

```
<!DOCTYPE html>
<html>
<head>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h2>Welcome to External CSS</h2>
</body>
</html>
```

Style.css:

```
h2 {
    color: blue;
    font-family: Arial;
}
```

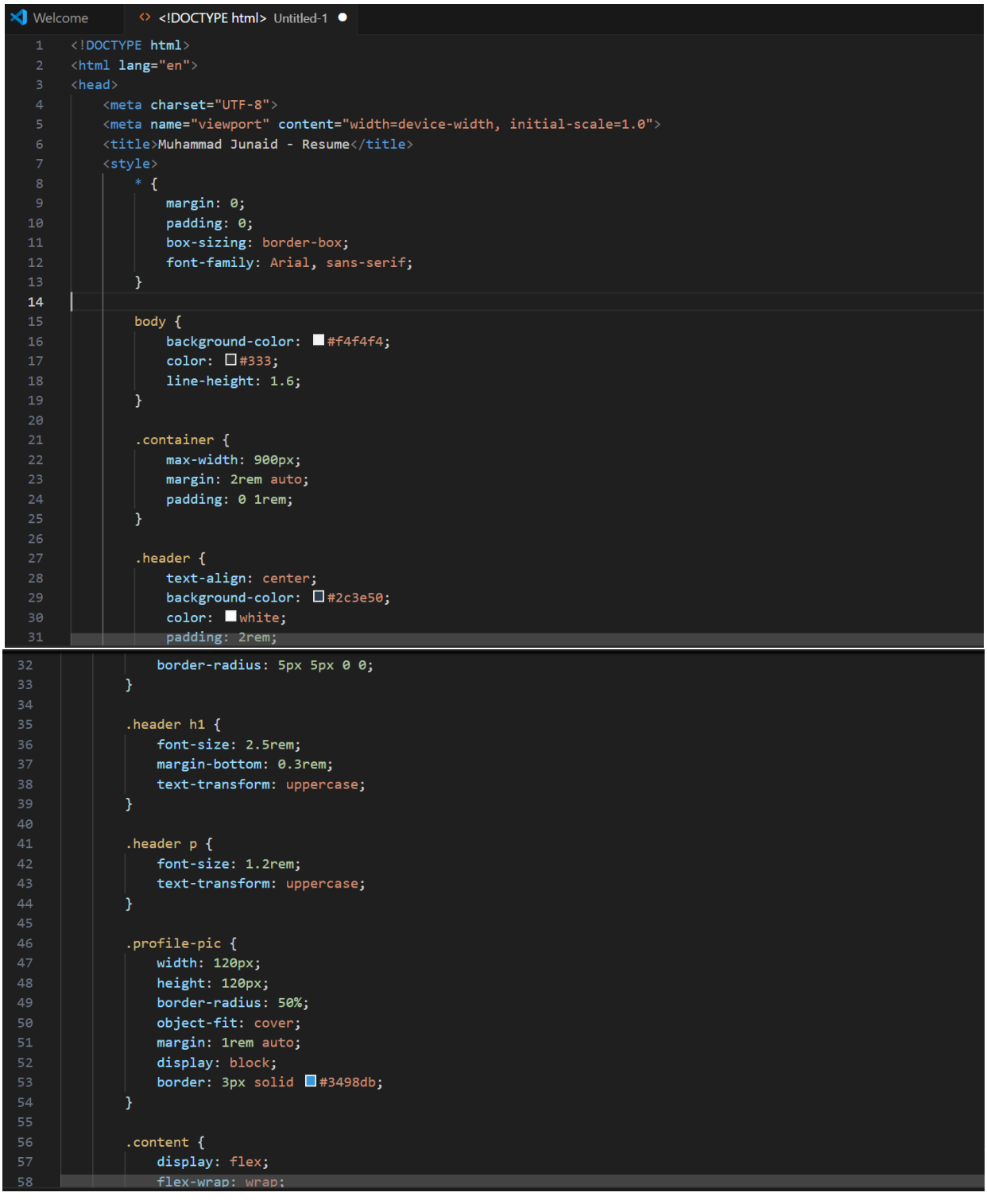
LAB-NO#10

Resume Webpage using HTML and CSS

❖Objective:

- A personal resume webpage using HTML for structure and CSS for styling. This project aims to showcase the ability to create a professional layout that displays personal, educational, and technical information in a clean and responsive format.

Code Screen Shot:



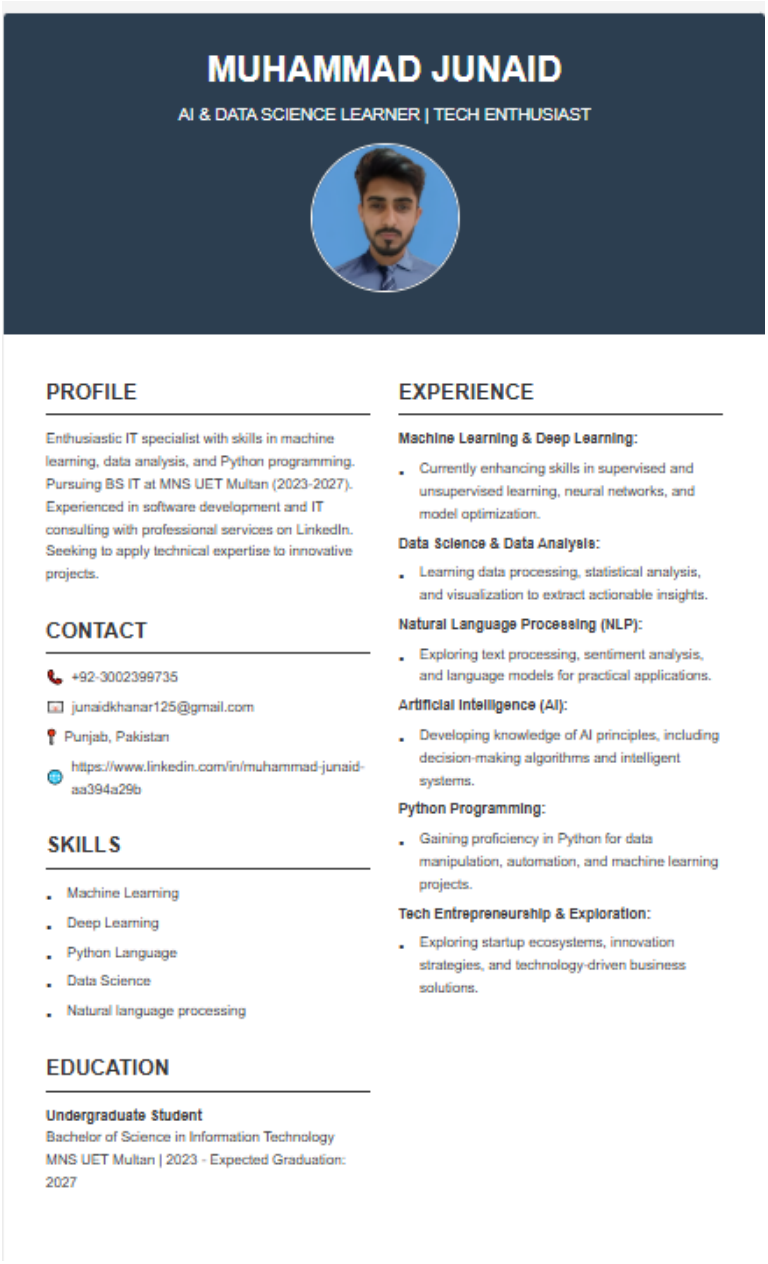
```
Welcome    <!DOCTYPE html> Untitled-1 ●
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Muhammad Junaid - Resume</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
            font-family: Arial, sans-serif;
        }

        body {
            background-color: #f4f4f4;
            color: #333;
            line-height: 1.6;
        }

        .container {
            max-width: 900px;
            margin: 2rem auto;
            padding: 0 1rem;
        }

        .header {
            text-align: center;
            background-color: #2c3e50;
            color: white;
            padding: 2rem;
            border-radius: 5px 5px 0 0;
        }

        .header h1 {
            font-size: 2.5rem;
            margin-bottom: 0.3rem;
            text-transform: uppercase;
        }

        .header p {
            font-size: 1.2rem;
            text-transform: uppercase;
        }

        .profile-pic {
            width: 120px;
            height: 120px;
            border-radius: 50%;
            object-fit: cover;
            margin: 1rem auto;
            display: block;
            border: 3px solid #3498db;
        }

        .content {
            display: flex;
            flex-wrap: wrap;
```

```
        background-color: white;
        padding: 2rem;
        border-radius: 0 0 5px 5px;
        box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
    }

    .left-column, .right-column {
        flex: 1;
        min-width: 300px;
        padding: 1rem;
    }

    .section {
        margin-bottom: 2rem;
    }

    .section h2 {
        font-size: 1.5rem;
        text-transform: uppercase;
        border-bottom: 2px solid #333;
        margin-bottom: 1rem;
        padding-bottom: 0.3rem;
    }

    .section p, .section li {
        font-size: 1rem;
        margin-bottom: 0.5rem;
    }

    .section ul {
        list-style: none;
    }

    .section ul li {
        position: relative;
        padding-left: 1.5rem;
    }

    .section ul li:before {
        content: "•";
        position: absolute;
        left: 0;
        color: #333;
        font-size: 1.2rem;
    }

    .contact-item {
        display: flex;
        align-items: center;
        margin-bottom: 0.5rem;
    }

    .contact-item span {
        margin-right: 0.5rem;
```

```
            font-weight: bold;
        }

        @media (max-width: 768px) {
            .content {
                flex-direction: column;
            }

            .left-column, .right-column {
                padding: 0.5rem;
            }

            .header h1 {
                font-size: 2rem;
            }

            .header p {
                font-size: 1rem;
            }
        }
    </style>
</head>
<body>
    <div class="container">
        <div class="header">
            <h1>Muhammad Junaid</h1>
            <p>AI & Data Science Learner | Tech Enthusiast</p>
            <img src="profile-pic.jpg" alt="Profile Picture" class="profile-pic">
        </div>
        <div class="content">
            <div class="left-column">
                <div class="section">
                    <h2>Profile</h2>
                    <p>Enthusiastic IT specialist with skills in machine learning, data analysis, and Python progra
                </div>
                <div class="section">
                    <h2>Contact</h2>
                    <div class="contact-item">
                        <span>📞</span> +92-3002399735
                    </div>
                    <div class="contact-item">
                        <span>✉️</span> junaidkhanar125@gmail.com
                    </div>
                    <div class="contact-item">
                        <span>📍</span> Punjab, Pakistan
                    </div>
                    <div class="contact-item">
                        <span>🌐</span> linkedin.com/in/muhammad-junaid-aa394a29b
                    </div>
                </div>
                <div class="section">
                    <h2>Skills</h2>
                    <ul>
                        <li>Machine Learning</li>
                        <li>Deep Learning</li>
                        <li>Python Language</li>
                        <li>Data Science</li>
                        <li>Natural language processing</li>
                    </ul>
                </div>
                <div class="section">
                    <h2>Education</h2>
                    <p><strong>Undergraduate Student</strong><br>
                    Bachelor of Science in Information Technology<br>
                    MNS UET Multan | 2023 - Expected Graduation: 2027</p>
                </div>
            </div>
            <div class="right-column">
                <div class="section">
                    <h2>Experience</h2>
                    <p><strong>Machine Learning & Deep Learning:</strong></p>
                    <ul>
                        <li>Currently enhancing skills in supervised and unsupervised learning, neural networks, and model optimization.</l
                    </ul>
                    <p><strong>Data Science & Data Analysis:</strong></p>
                    <ul>
                        <li>Learning data processing, statistical analysis, and visualization to extract actionable insights.</li>
                    </ul>
                    <p><strong>Natural Language Processing (NLP):</strong></p>
                    <ul>
```

```
                        <li>Exploring text processing, sentiment analysis, and language models for practical applications.</li>
                    </ul>
                    <p><strong>Artificial Intelligence (AI):</strong></p>
                    <ul>
                        <li>Developing knowledge of AI principles, including decision-making algorithms and intelligent systems.</li>
                    </ul>
                    <p><strong>Python Programming:</strong></p>
                    <ul>
                        <li>Gaining proficiency in Python for data manipulation, automation, and machine learning projects.</li>
                    </ul>
                    <p><strong>Tech Entrepreneurship & Exploration:</strong></p>
                    <ul>
                        <li>Exploring startup ecosystems, innovation strategies, and technology-driven business solutions.</li>
                    </ul>
                </div>
            </div>
        </div>
    </div>
</body>
</html>
```

Output:



LAB-NO#11

CSS Selectors

❖ Objective:

- The goal of this lab is to learn how to use CSS selectors to apply styles to specific HTML elements. Selectors help us decide which parts of a webpage should be styled.

What is a CSS Selector?

A CSS selector is a way to target specific elements in your HTML so you can change their appearance using CSS. For example, if you want to make all paragraphs red or center-align a heading, you use selectors to tell CSS where to apply those styles.

Explanation of CSS Selectors (in simple terms):

There are many types of selectors in CSS, but in this lab we'll focus on the most basic and commonly used ones:

1. Element Selector

This is the most basic type of selector. It targets all HTML elements of the same type. For example, if you write:

```
p {
  color: green;
}
```

- It means: "Make the text color of all <p> tags green."
- It will apply the style to every paragraph in the webpage.

2. Class Selector

Class selectors are used when you want to style some elements, not all. You first give the element a class in HTML, then you use . (dot) in CSS to apply the style.

Example:

```
.highlight {
  background-color: yellow;
  font-weight: bold;
```

}

So only this paragraph will have a yellow background and bold text. Other paragraphs won't be affected.

3. ID Selector

If there's one unique element on the page and you want to style only that one, then you use an ID selector. In CSS, you use # before the ID name.

Example:

```
#main-heading {
  text-align: center;
  color: blue;
}
```

This style will apply only to this <h1> tag. IDs must be unique — they should not be reused on other elements.

4. Universal Selector

Sometimes you want to apply a style to every element on the page. For that, CSS provides the * selector — known as the universal selector.

Example:

```
* {
  font-family: Arial, sans-serif;
}
```

This means: "Apply Arial font to every element on the page."

5. Group Selector

This selector is used when you want to apply the same style to multiple different elements at once. You just write their names separated by commas.

Example:

```
h1, p, div {
  padding: 10px;
}
```

Now, all <h1>, <p>, and elements will get 10 pixels of padding.

Complete Code Example:

Here is a simple web page that uses all the above selectors:

```
<!DOCTYPE html>
<html>
<head>
   <title>CSS Selectors Lab</title>
   <style>
      h1 {
         color: green;
      }
      .highlight {
         background-color: yellow;
         font-weight: bold;
      }
      #main-heading {
         text-align: center;
         color: blue;
      }
      * {
         font-family: Arial, sans-serif;
      }
      p, div {
         border: 1px solid gray;
         padding: 10px;
      }
   </style>
</head>
<body>
   <h1 id="main-heading">Welcome to CSS Selectors Lab</h1>
   <p>This is a normal paragraph.</p>
   <p class="highlight">This is a highlighted paragraph using class selector.</p>
   <div>This is a div styled with group selector.</div>
</body>
</html>
```

What You Learned:

- CSS selectors help you control which HTML elements are styled.
- You can target elements using tag names, classes, IDs, or even all elements.
- Classes can be used many times; IDs should only be used once per page.
- You can apply the same style to many tags at once using a group selector.

LAB-NO#12

CSS Box Model

❖ Objective:

- To understand the CSS box model and how different parts like margin, border, padding, and content affect the layout and spacing of elements.

What is the CSS Box Model?

In CSS, every HTML element is treated like a box. This box is made up of four parts, which are:

- Content – The actual text or image inside the element.
- Padding – The space between the content and the border.
- Border – A line around the padding (and content).
- Margin – The space outside the border that separates the element from other elements.

Imagine a gift box:

- The gift inside = content
- The bubble wrap around the gift = padding
- The wrapping paper = border
- The space between this gift and others in the room = margin

Explanation:

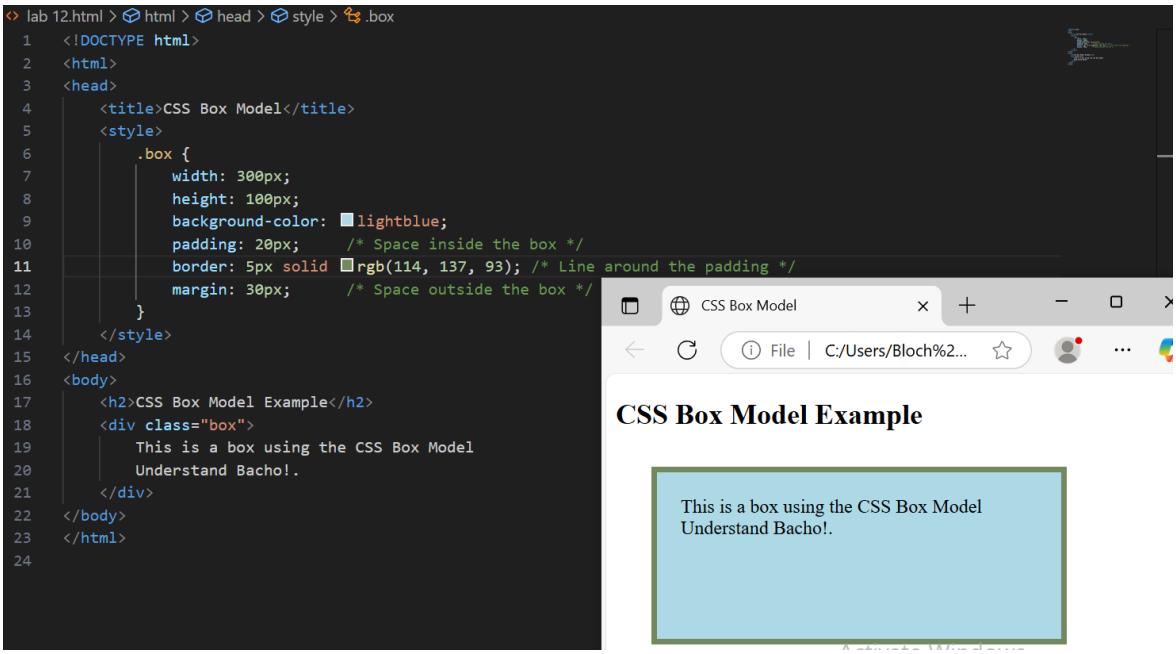
The box model helps us control the spacing inside and around HTML elements. This is very important when designing webpages because it helps us keep everything neat and spaced out properly.

Let's say you have a paragraph:

```
<p>This is a simple paragraph.</p>
```

Using the box model, we can control how close this paragraph is to other content, how thick its border is, and how much space is around the text inside it.

Complete Example (HTML + CSS):



Explanation of What This Code Does:

- **width and height:** set the size of the content area.
- **padding:** adds space inside the box (between the text and the border).
- **border:** draws a solid line around the box.
- **margin:** adds space outside the box to keep it away from other elements.

So this code creates a light blue box with:

- Space inside the text (padding),
- A green border around it,
- Space outside the border (margin).

What You Learned:

- How the CSS box model works
- How to control element spacing using padding, border, and margin

- How box sizing affects the layout of your webpage

LAB-NO#13

CSS Positioning

❖ Objective:

- To understand how to position elements on a webpage using different CSS positioning properties like static, relative, absolute, and fixed.

What is positioning in CSS?

CSS positioning allows you to control the exact placement of an element on the webpage. By default, elements appear one after another (top to bottom), but using CSS position properties, you can move them up, down, left, or right as needed.

There are four main types of positioning in CSS:

1. Static (default)
2. Relative
3. Absolute
4. Fixed

Explanation:

1. Static Position:

This is the default position. All elements are placed in the normal flow of the document.

```
div {
  position: static;
}
```

You usually don't need to write this because it's already the default.

2. Relative Position:

The element stays in its normal position, but you can move it slightly using top, left, right, or bottom.

```
.box {
  position: relative;
  top: 20px;
  left: 30px;
}
```

This means: “Move the box 20px down and 30px to the right from its original position.”

3. Absolute Position:

The element is removed from the normal flow and placed exactly where you want it relative to the nearest positioned ancestor (or the page if none).

```
.box {
  position: absolute;
  top: 50px;
  left: 100px;
}
```

This places the box exactly 50px from the top and 100px from the left of its container or page.

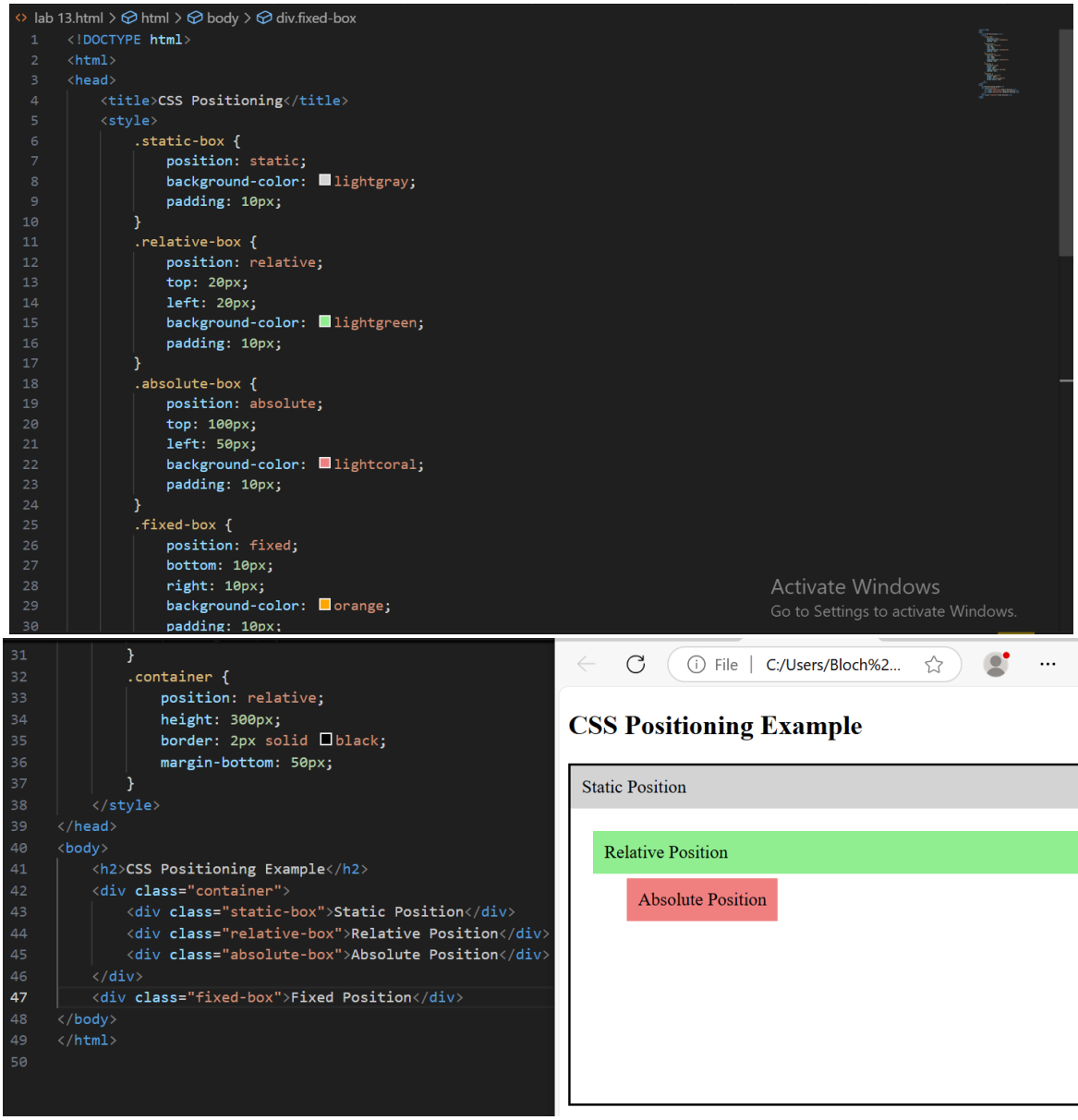
4. Fixed Position:

This element is always fixed on the screen — it does not move when you scroll. Useful for headers, footers, or floating buttons.

```
.box {
  position: fixed;
  bottom: 10px;
  right: 10px;
}
```

Now the box will stay stuck to the bottom-right corner of the screen.

Complete Example (HTML + CSS):



```
lab 13.html > html > body > div.fixed-box
<!DOCTYPE html>
<html>
<head>
    <title>CSS Positioning</title>
    <style>
        .static-box {
            position: static;
            background-color: lightgray;
            padding: 10px;
        }
        .relative-box {
            position: relative;
            top: 20px;
            left: 20px;
            background-color: lightgreen;
            padding: 10px;
        }
        .absolute-box {
            position: absolute;
            top: 100px;
            left: 50px;
            background-color: lightcoral;
            padding: 10px;
        }
        .fixed-box {
            position: fixed;
            bottom: 10px;
            right: 10px;
            background-color: orange;
            padding: 10px;
        }
        .container {
            position: relative;
            height: 300px;
            border: 2px solid black;
            margin-bottom: 50px;
        }
    </style>
</head>
<body>
    <h2>CSS Positioning Example</h2>
    <div class="container">
        <div class="static-box">Static Position</div>
        <div class="relative-box">Relative Position</div>
        <div class="absolute-box">Absolute Position</div>
    </div>
    <div class="fixed-box">Fixed Position</div>
</body>
</html>
```

Activate Windows
Go to Settings to activate Windows.

File | C:/Users/Bloch%2...

CSS Positioning Example

Static Position

Relative Position

Absolute Position

What You Learned:

- **Static:** default flow of elements
- **Relative:** move an element from its normal place
- **Absolute:** place element exactly where you want it, removed from normal flow
- **Fixed:** stick element to a fixed position on screen

LAB-NO#14

JavaScript Basics – Variables and Data Types

❖ Objective:

- To understand how to create variables in JavaScript and use different data types like numbers, strings, and booleans.

What is JavaScript?

JavaScript is a programming language used to make websites interactive. It can change HTML content, handle user input, perform calculations, and much more — directly in the browser.

What is a Variable?

A variable is like a container that stores a value. You can name it and put something inside — like text or a number — and use it later.

How to Declare a Variable in JavaScript:

There are 3 ways to create variables:

1. **var** – old way (not used much now)
2. **let** – used for values that might change
3. **const** – used for values that do not change

Example:

```
let name = "Junaid";

const age = 20;

var city = "Multan";
```

JavaScript Data Types:

- **String** – Text in quotes

Example: "Hello"

Use for names, messages, etc.

- **Number** – Numeric value

Example: 10, 3.14

Use for age, scores, prices.

- **Boolean** – True or false

Example: true, false

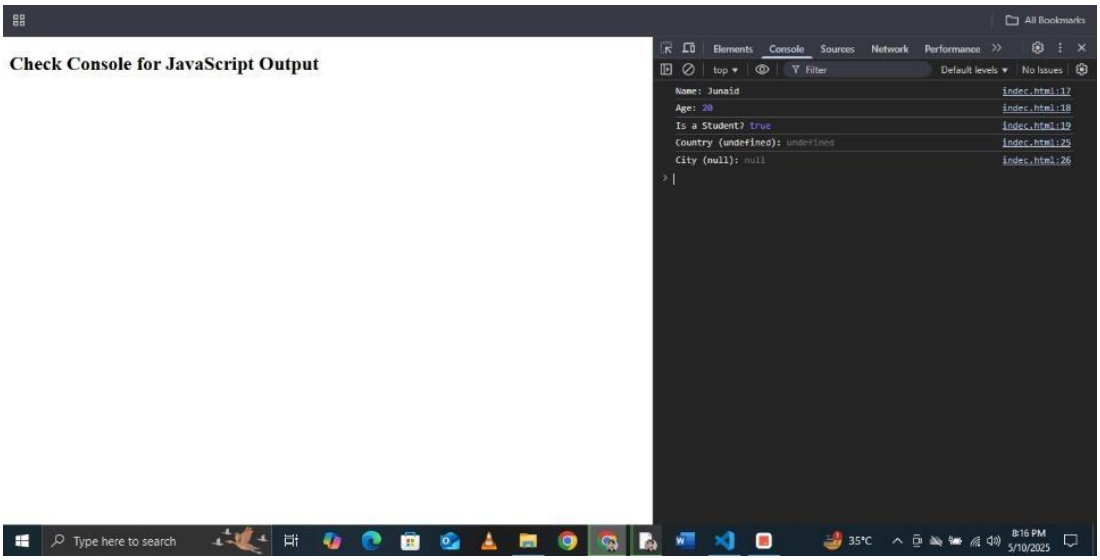
Useful for conditions and logic.

- **Undefined** – A variable that is declared but not given a value.
- **Null** – Intentionally empty value.

Code Example:

```
Web Technology Lab Work > indec.html > html > body > script
<!DOCTYPE html>
<html>
<head>
    <title>JavaScript Variables and Data Types</title>
</head>
<body>

    <h2>Check Console for JavaScript Output</h2>

    <script>
        // Declaring variables
        let username = "Junaid";
        const age = 20;
        var isStudent = true;

        // Showing results in console
        console.log("Name:", username);
        console.log("Age:", age);
        console.log("Is a Student?", isStudent);

        // Undefined and null
        let country;
        let city = null;

        console.log("Country (undefined):", country);
        console.log("City (null):", city);
    </script>

</body>
</html>
```

Output:



What You Learned:

- JavaScript is used for interactive web features.
- A variable stores information.
- Data types include strings, numbers, booleans, null, and undefined.
- let and const are the modern ways to declare variables.

LAB-NO#15

JavaScript Functions and Events

❖ Objective:

➢ To understand how to create functions in JavaScript and how to use events to make your webpage interactive (like clicking a button to show a message).

What is a Function in JavaScript?

A function is a reusable block of code that performs a specific task. You can call it whenever you need it.

Think of it like a machine:

- You give it input (if needed),
- It does something,
- You get the output.

How to Create a Function:

```
function greetUser() {
   alert("Hello, welcome to Junaid's site!");
}
```

To call (run) this function, write:

```
greetUser();
```

What are Events in JavaScript?

An event is something that happens in the browser — like when the user clicks a button, moves the mouse, or types in a textbox.

You can write JavaScript that runs when an event happens.

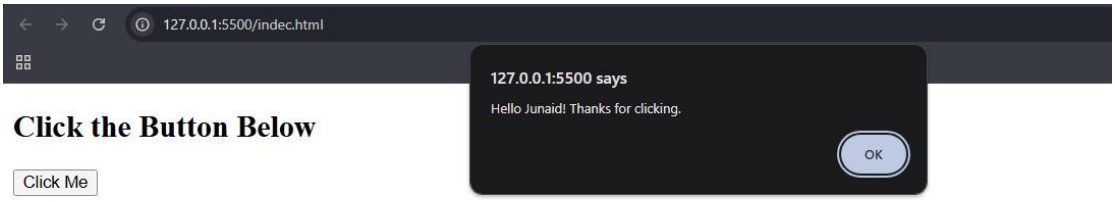
Example: When someone clicks a button, show a message.

Code Example:

```
Web Technology Lab Work > indec.html > html
<!DOCTYPE html>
<html>
<head>
    <title>JavaScript Functions and Events</title>
</head>
<body>

    <h2>Click the Button Below</h2>
    <button onclick="greetUser()">Click Me</button>

    <script>
        // Function to show alert message
        function greetUser() {
            alert("Hello Junaid! Thanks for clicking.");
        }
    </script>

</body>
</html>
```

Output:

After clicking the button:



What You Learned:

- A function is a set of instructions you can reuse.
- JavaScript events respond to user actions like clicking.
- You can use onclick to connect a button to a function.

LAB-NO#16

JavaScript DOM Manipulation

❖ Objective:

- To understand how to access and change HTML elements using JavaScript with the help of the DOM (Document Object Model).

What is the DOM?

The DOM (Document Object Model) is a structure that the browser creates from your HTML page. It turns every element (like headings, paragraphs, buttons) into objects that JavaScript can interact with.

With JavaScript, you can:

- Read content from an element
- Change text, styles, and attributes
- Add or remove elements from the page

Common DOM Methods:

- **document.getElementById("id")** – Get an element by its ID
- **innerHTML** – Read or change the HTML content
- **style.property** – Change the CSS style of an element

Complete Example: Change Text and Style with Button Click

Web Technology Lab Work > index.html > html > body > script

```html
<!DOCTYPE html>
<html>
<head>
    <title>JavaScript DOM Manipulation</title>
</head>
<body>

    <h2 id="heading">Welcome to My Web Page</h2>
    <button onclick="changeContent()">Change Heading</button>

    <script>
        function changeContent() {
            // Get the element by its ID
            let headingElement = document.getElementById("heading");

            // Change the text
            headingElement.innerHTML = "Hello Junaid! This is updated.";

            // Change the color and font size
            headingElement.style.color = "blue";
            headingElement.style.fontSize = "28px";
        }
    </script>
</body>
</html>
```

Output:

What You Learned:

- The DOM lets JavaScript interact with your webpage.
- You can get elements using getElementById.
- You can change content with .innerHTML.
- You can style elements with .style.property.